

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

Programmation dynamique côté serveur avec PHP

Module 1 : Programmation

Consigne : Les exercices suivants mobilisent l'ensemble des savoirs du chapitre. Vous devez proposer une architecture de code cohérente, justifier vos choix techniques et analyser les failles potentielles. Aucun code complet n'est attendu, mais des extraits significatifs et structurés.

EXERCICE 1 Assistant d'inscription multi-étapes (Wizard)

On souhaite concevoir un script unique `inscription.php` gérant un formulaire en 3 étapes : 1. Identifiants (email, mot de passe), 2. Profil (nom, date naissance), 3. Confirmation. La navigation se fait par boutons « Précédent »/ « Suivant ». Les données sont conservées sans base de données ni sessions PHP (non au programme), uniquement via des champs cachés (`<input type="hidden">`) réinjectés à chaque étape.

1. **Architecture des données :** Proposez la structure d'un tableau associatif `$donnees` centralisant toutes les valeurs du formulaire. Comment initialisez-vous ce tableau en fusionnant `$_POST` et les valeurs par défaut ? Justifiez l'usage de l'opérateur de fusion null (`??`) ou de `isset()` pour éviter les avertissements `Undefined array key`.
2. **Validation modulaire :** Écrivez l'en-tête (signature) et le docblock d'une fonction `validerEtape(int $etape, array $donnees): array` renvoyant un tableau associatif d'erreurs (clé = nom du champ, valeur = message). Expliquez comment vous utilisez les opérateurs de comparaison et logiques pour valider : format email (`filter_var`), longueur mot de passe (≥ 8), cohérence date naissance (mineur?).
3. **Gestion de l'état et navigation :** Décrivez l'algorithme (pseudo-code ou PHP) déterminant l'étape courante (`$etapeActuelle`) en analysant `$_POST['action']` (suivant, precedent, confirmer) et la validité de l'étape précédente. Comment gérez-vous la ré-injection des données dans les champs cachés du formulaire HTML suivant ? Donnez un exemple de boucle `foreach` sur `$donnees` pour générer ces champs.
4. **Sécurité (Analyse critique) :** Identifiez deux failles majeures inhérentes à cette technique de persistance par champs cachés (ex : falsification de données, exposition de données sensibles). Proposez une contrainte de conception (sans sessions) pour atténuer l'une d'elles.

EXERCICE 2 Module de téléversement et catalogage d'images

On développe `upload.php` recevant un fichier via `$_FILES['avatar']`. Le script doit : vérifier l'extension (jpg, png, webp), la taille (< 2 Mio), le type MIME réel, générer un nom unique, déplacer le fichier vers `/var/www/uploads/`, et enregistrer les métadonnées (nom original, nom stocké, taille, date, IP client) dans un fichier `catalogue.json` (tableau JSON

d'objets).

1. **Superglobales et environnement** : Comment accéder à l'adresse IP du client ? Donnez l'instruction exacte utilisant `$_SERVER`. Pourquoi `$_SERVER['HTTP_X_FORWARDED_FOR']` peut-elle être nécessaire et risquée ?
2. **Sécurisation du fichier** : Détaillez les 4 vérifications successives (erreur `UPLOAD_ERR_OK`, taille, extension via `pathinfo()`, MIME via `mime_content_type()`) avant tout déplacement. Pourquoi l'extension seule est-elle insuffisante ? Écrivez la condition logique complète (opérateurs `&&`, `||`, `!`) pour autoriser le traitement.
3. **Stockage structuré** : Proposez la structure d'un tableau associatif `$meta` pour une entrée du catalogue. Montrez comment lire le fichier JSON existant (`file_get_contents`, `json_decode`), y ajouter `$meta`, puis ré-écrire le fichier (`json_encode` avec `JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES`, `file_put_contents`). Gérez le cas où le fichier n'existe pas encore.
4. **Fonctionnalité transversale** : Écrivez la déclaration d'une fonction `genererNomUnique(string $extension): string` combinant `uniqid()`, `bin2hex(random_bytes(4))` et l'extension. Expliquez pourquoi `move_uploaded_file()` est impératif plutôt que `copy()` ou `rename()`.

EXERCICE 3 Mini-routeur MVC "Front Controller" (Synthèse)

L'application utilise un point d'entrée unique `index.php` recevant toutes les requêtes. L'URL appelée est `index.php?route=controleur/action¶m=valeur`. Le script doit instancier le bon contrôleur (fichier `contrôleurs/Contrôleleur.php` contenant une classe `Contrôleleur` avec une méthode `action()`), lui passer les paramètres `$_GET` et `$_POST` fusionnés, et afficher la vue retournée (fichier `vues/contrôleleur_action.php`) par inclusion.

1. **Routing et constantes** : Définissez les constantes `RACINE`, `DOSSIER_CONTRÔLEURS`, `DOSSIER_VUES` via `define()` en utilisant `__DIR__`. Écrivez l'algorithme d'extraction du nom du contrôleur et de l'action depuis `$_GET['route']` (ex : `utilisateur/profil` → `Contrôleleur = Utilisateur`, `Action = profil`). Gérez l'absence de route (défaut : `accueil/index`).
2. **Inclusion dynamique et portée** : Le contrôleur est inclus via `require_once`. La vue est incluse via une fonction `renduVue(string $nomVue, array $donnees): void`. Expliquez la portée des variables `$donnees` à l'intérieur du fichier de vue inclus. Comment transformer les clés du tableau `$donnees` en variables locales dans la vue (indice : `extract()`) ? Quels risques (collision, sécurité) ?
3. **Séparation des responsabilités** : Le contrôleur `Article` possède une méthode `lister()` qui prépare un tableau d'articles (simulé) et appelle `renduVue('article_lister', ['articles' => $articles])`. Dans la vue `article_lister.php`, on affiche un tableau HTML. Montrez comment utiliser `foreach`

et la syntaxe alternative (`endforeach`) pour itérer. Pourquoi ne doit-on jamais faire de `echo` ou `print` directement dans le contrôleur ?

4. Analyse d'architecture (Ouvert) : Critiquez cette architecture simplifiée au regard de :
1. La gestion des erreurs 404 (contrôleur/vue inexistant).
 2. La protection contre l'inclusion de fichiers arbitraires (LFI) via le paramètre `route`.
 3. L'impossibilité de faire de la redirection HTTP (`header('Location: ...')`) après un `POST` (PRG Pattern) si du HTML a déjà été envoyé par la vue. Proposez une piste pour le point 3.